

Full Lecture Note - Tutorial 02 Shader Colors

Source file: `tutorials/02_shader_colors.py`

1. Lesson Goal

This program teaches how color is passed from the vertex stage to the fragment stage, and how the GPU interpolates that color across triangle surfaces.

The core idea is: each vertex has a color, and pixels between vertices receive blended colors automatically.

2. What This File Builds

The file builds a minimal OpenGL pipeline with:

- A Pygame window with OpenGL context
- A vertex shader and fragment shader
- One VBO containing position and color together
- Rendering using `glDrawArrays(GL_TRIANGLES, 0, 9)`

Visual result:

- White background
- 3 colorful triangles
- Smooth gradient transitions inside each triangle

3. Program Flow (High-Level)

Execution order:

1. Program starts in `if __name__ == "__main__":`
2. `TestContext()` object is created
3. `main_loop()` runs
4. `init_display()` creates window and projection
5. `OnInit()` compiles shaders and creates VBO
6. Repeated frame loop:
 - `handle_events()` for input
 - `Render()` draws geometry
 - `pygame.display.flip()` swaps buffers

4. Detailed Walkthrough by Function

4.1 `__init__(self, width=800, height=600)`

- Stores window size
- Sets `self.running = True` for loop control

No OpenGL work happens here.

4.2 `init_display(self)`

This function creates and configures the rendering context.

What happens:

- `pygame.init()` initializes SDL systems
- `pygame.display.set_mode(..., DOUBLEBUF | OPENGGL)` creates OpenGL window with double buffering
- Window title set to Tutorial 02
- `glClearColor(1,1,1,1)` sets white background

Projection and camera setup:

- Switch to projection matrix: `glMatrixMode(GL_PROJECTION)`
- Reset matrix: `glLoadIdentity()`
- Perspective projection via `gluPerspective(45, aspect, 0.1, 50.0)`
- Switch to model-view matrix and reset
- `glTranslatef(0,0,-10)` moves scene away from camera so geometry is visible

4.3 `OnInit(self)`

This is the most important setup stage.

A. Shader compile error demonstration

The code intentionally compiles broken shader code:

```
" void main() { "
```

Purpose:

- Show students real compile error behavior
- Confirm exception handling works

B. Vertex shader

Key shader lines:

- `varying vec4 vertex_color;`
- `gl_Position = gl_ModelViewProjectionMatrix * gl_Vertex;`
- `vertex_color = gl_Color;`

Meaning:

- Vertex position is transformed to clip space
- Vertex color enters pipeline through `gl_Color`
- Color is exported as `vertex_color` varying for interpolation

C. Fragment shader

Key shader lines:

- `varying vec4 vertex_color;`
- `gl_FragColor = vertex_color;`

Meaning:

- Fragment shader receives interpolated color for current fragment
- Writes that color to framebuffer

D. Program link

- `self.shader = shaders.compileProgram(vertex, fragment)`

This links both shader stages into one executable GPU program.

E. VBO creation

The VBO stores packed records of 6 floats each:

- Position: `x, y, z`
- Color: `r, g, b`

Layout per vertex:

`[x, y, z, r, g, b]`

Data contains 9 vertices (3 triangles total).

4.4 `Render(self)`

This runs every frame.

Frame operations:

1. `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`
2. Activate shader with `glUseProgram(self.shader)`
3. Bind VBO
4. Enable fixed-function input arrays:
 - `GL_VERTEX_ARRAY`
 - `GL_COLOR_ARRAY`
5. Define memory interpretation:
 - `glVertexPointer(3, GL_FLOAT, 24, self.vbo)`
 - `glColorPointer(3, GL_FLOAT, 24, self.vbo+12)`
6. Draw: `glDrawArrays(GL_TRIANGLES, 0, 9)`
7. Unbind and disable states
8. `glUseProgram(0)` cleanup

Important memory details:

- 1 float = 4 bytes
- 6 floats per vertex = 24-byte stride

- Color starts after first 3 floats = 12-byte offset

4.5 `handle_events(self)`

- Reads Pygame event queue
- Close button or `ESC` sets `self.running = False`

4.6 `main_loop(self)`

- Calls `init_display()` and `OnInit()` once
- Creates `clock`
- Repeats until exit:
 - input
 - render
 - buffer swap
 - frame cap at 60 FPS
- Calls `pygame.quit()` at end

5. Why the Gradient Appears

Each triangle vertex has a different RGB value. The rasterizer computes weighted blends for fragments inside the triangle.

Mathematically, if weights are barycentric values w_1, w_2, w_3 with $w_1 + w_2 + w_3 = 1$, interpolated color is:

$$C = w_1 C_1 + w_2 C_2 + w_3 C_3$$

This interpolation is done by the GPU automatically when a varying is used.

6. Concepts Students Should Understand

- Vertex attributes are per-vertex input data.
- Varyings carry data vertex -> fragment.
- Fragment stage runs for many fragments, not just one per triangle.
- Shader program must be active before drawing.
- Pointer stride and offset must match actual VBO layout.

7. Common Student Errors

1. Wrong stride/offset in `glVertexPointer` or `glColorPointer`.
2. Forgetting `glEnableClientState(GL_COLOR_ARRAY)`.
3. Shader compile errors from syntax mismatches.
4. Using wrong vertex count in `glDrawArrays`.

8. Practical Summary

This file is a complete example of color interpolation using GLSL varying values with packed VBO data. Students should leave this lesson understanding data flow from CPU -> vertex shader -> interpolation -> fragment shader -> final pixel.